



Exploración de Estilos Arquitectónicos y Stack Tecnológico

Grupo 4 – Presentación 1, Documento Técnico

Arquitectura de Software

Andrés Armando Sánchez Martín

Equipo 1

Empresa Vitalis - Proyecto RIBAS: Red Interinstitucional de Bancos de Sangre

Septiembre, 2026

TABLA DE CONTENIDO

TABLA DE CONTENIDO	1
INVESTIGACIÓN	2
Investigación del Estilo Arquitectónico: Monolito	3
Investigación de Stack Asignado	9
ANÁLISIS ARQUITECTÓNICO	15
Matriz de Atributos de Calidad vs. Estilo (Monolito en capas)	16
Matriz de Análisis de Principios vs. Estilo (Monolito en capas)	17
Matriz de Análisis de Tácticas vs. Estilo y Stack	18
Matriz de análisis de mercado Laboral vs estilo y stack	21
DISEÑO EJEMPLO PRÁCTICO	23
Diagrama de alto nivel (HDL)	24
Diagrama de contexto (C1)	24
Diagrama de contenedores (C2)	25
Diagrama dinámico (Préstamo y devolución)	26
Diagrama de despliegue	27
LECCIONES APRENDIDAS	28
REFERENCIAS	29

INVESTIGACIÓN

Investigación del Estilo Arquitectónico: Monolito

Definición clara

I. ¿Qué es?

La arquitectura monolítica es un modelo tradicional para diseñar programas de software. En el desarrollo, un modelo de esta arquitectura es uno en el que se usa un solo código base para realizar múltiples funciones de una aplicación o programa, si es que no todas sus funciones.

De acuerdo con lo anterior se puede decir que un monolito no es más que una aplicación diseñada y desplegada como una sola unidad indivisible. Todos los componentes de esta, ya sea el almacenamiento de los datos, la lógica del negocio, interfaz del cliente o el servidor, se encuentran juntos, empaquetados en un solo lugar y gestionándose como una sola unidad. Normalmente esas funcionalidades se encuentran fuertemente acopladas en este tipo de arquitectura.

II. ¿Qué no es?

Algunas de las cosas que las personas suelen asumir incorrectamente del monolito son:

1. Es un código desordenado

Un código desordenado y sin estructura clara se conoce en el mundo de software como Big Ball of Mud, y puede ocurrir en cualquier arquitectura de las conocidas, no solo en la monolítica. Un monolito puede estar bien organizado, con sus capas claramente separadas, o puede terminar siendo un desastre, esto depende de las personas que lo desarrollen, mas no del estilo arquitectónico en sí.

2. No tiene organización interna

Que una aplicación se despliegue como una sola unidad no significa que su código esté mezclado y sin orden. Internamente, un monolito puede estar dividido en capas o módulos bien definido, lo único que lo define es que se construye y despliega en un solo paquete.

3. Es una aplicación pequeña

Empresas grandes que atienden a muchísimos usuarios como Shopify o Stack Overflow usan una arquitectura monolítica. Esto demuestra que un monolito bien diseñado puede escalar y soportar cargas altas de trabajo, sin necesidad de recurrir a microservicios u a otra arquitectura similar.

Clasificación del estilo

Dentro de la arquitectura hay diferentes variaciones reconocidas dentro de la industria, entre esas se encuentran:

- **Layered:** Organización por capas (presentación, lógica y datos).
- **Modular:** organizado por dominio de negocio, bajo acoplamiento y fronteras estrictas entre módulos.
- **Big Ball of Mud:** sin organización y con alto acoplamiento.

Para el presente trabajo se ha decidido ubicar el sistema como un monolito “layered” o por capas, este es el más común y el que encaja mejor con Spring Boot y Thymeleaf, el stack tecnológico que se debe de manejar.

Características principales

Dentro de las características principales de la arquitectura monolítica encontramos:

- **Base de código única:** Todo el programa se desarrolla y mantiene con una sola base de código.
- **Almacenamiento centralizado:** Normalmente se hace uno de una sola base de datos con la cual interactúan todas las partes del sistema.
- **Despliegue unificado:** El programa se construye y despliega como una sola entidad, cualquier cambio que se realice requiere volver a compilar y volver a realizar el despliegue de todo.
- **Comunicación síncrona entre componentes:** Normalmente esta comunicación ocurre por llamadas directas a métodos o funciones. Se pueden llamar directamente sin necesitar un proceso intermedio.
- **Espacio de memoria compartido:** Los componentes del programa corren en el mismo proceso y las funciones de este pueden llamarse directamente sin necesidad de comunicación entre procesos

Historia y evolución

La arquitectura monolítica no es un invento reciente, sino el punto de partida histórico del desarrollo de software empresarial. Hasta finales de los años 80, aplicaciones como la nómina, el control de inventario o la contabilidad se ejecutaban en mainframes, que eran computadoras grandes y costosas a las que se accedía mediante terminales sin capacidad de procesamiento propio. En ese momento no existían redes rápidas ni arquitecturas distribuidas, por lo que construir todo en un solo lugar no era una opción entre varias, sino la única forma viable de desarrollar software; con la llegada de la web, las aplicaciones empresariales migraron a modelos cliente-servidor y luego a aplicaciones.

Con el crecimiento de internet y la necesidad de escalar sistemas muy grandes, surgieron alternativas como la arquitectura orientada a servicios (SOA) y, más adelante, los microservicios, impulsados por empresas como Netflix y Amazon. Sin embargo, el monolito nunca desapareció: muchas empresas que adoptaron microservicios se encontraron con una complejidad operativa muy alta, entre servicios que coordinar, redes que administrar y costos

de infraestructura elevados, por lo que en los últimos años ha resurgido el llamado "monolito modular", un enfoque que aprovecha la simplicidad de desplegar todo en un solo lugar, pero organizando el código internamente en módulos bien definidos por dominio de negocio.

Ventajas y desventajas

I. Ventajas

- **Fácil despliegue:** un solo ejecutable o directorio facilita el despliegue, desarrollo más simple al basarse en una sola base de código.
- **Rendimiento:** la interacción entre componentes es directa y rápida porque ocurre dentro del mismo proceso.
- Las pruebas end-to-end suelen ser más simples porque todos los componentes están disponibles y se prueban juntos en un mismo entorno.

II. Desventajas

- Un cambio de código en un componente obliga a que se deba actualizar y volver a desplegar los componentes relacionados, lo que puede ser lento en productos complejos o equipos grandes.
- Escalabilidad limitada: escalar un componente específico es difícil porque hay que escalar toda la aplicación, sin embargo si un componente falla, todo el sistema puede caerse.
- A medida que un proyecto crece, se vuelve más difícil que equipos grandes trabajen de forma simultánea.

Problemas comunes que se presentan y patrones

Dentro de la arquitectura monolítica se pueden presentar varios problemas si no se manejan de forma correcta, algunos de los problemas son los siguientes:

- **Big Ball of Mud:** es el problema más común y se da cuando el sistema se convierte en una masa de código desorganizado, sin una estructura real. Esto pasa normalmente porque se van acumulando funcionalidades con el paso del tiempo sin diseñar cómo deben interactuar entre sí de forma modular y mantenible. En este problema, cualquier módulo puede comunicarse con cualquier otro sin ningún control, lo que genera una explosión de dependencias y termina en lo que se conoce como código espagueti.
- **Acoplamiento excesivo entre módulos:** cuando las distintas partes del sistema no están claramente separadas, un cambio pequeño en un módulo puede afectar a otros módulos que no deberían verse involucrados.
- **Dificultad para mantener el orden a medida que el sistema crece:** entre más funcionalidades se le agregan al monolito, más difícil se vuelve entender el flujo completo del sistema si no se sigue una estructura clara desde el inicio.

Para evitar estos problemas, el patrón que se utiliza es la Arquitectura en Capas, en la cual cada componente solo puede interactuar con la capa que tiene inmediatamente arriba o abajo, una capa le presta servicios a la capa superior y le pide servicios a la capa inferior.

Identificar patrones aplicables y cuando usarlos

Los diferentes patrones que se pueden aplicar y en que ocasiones se pueden utilizar son los siguientes:

- **Arquitectura en capas:** cuando se quiere separar las responsabilidades sin tener una gran complejidad como en servicios distribuidos.
- **Monolito modular:** cuando el equipo o el dominio crece y se quiere aislar los módulos del negocio dentro del mismo despliegue, sin llegar a microservicios.
- Evitar el Big Ball of Mud manteniendo disciplina en capas y responsabilidades desde el inicio.

Tabla 1

Matriz Comparativa de Tipos de Monolito

Tipo de Monolito	Definición	Características principales	Ventajas	Desventajas	Ejemplo / Cuándo usarlo
Monolito Singular (tradicional / single-process)	Toda la aplicación (interfaz, lógica de negocio y acceso a datos) corre en un único proceso y se despliega como una sola unidad indivisible, sin división interna clara.	Un solo ejecutable, un solo repositorio, sin separación intencional entre capas o dominios.	Muy simple de construir y desplegar al inicio de un proyecto.	Tiende rápidamente al Big Ball of Mud si el equipo o el proyecto crecen, ya que no hay fronteras internas.	Proyectos muy pequeños, prototipos o MVPs con pocos desarrolladores.
Monolito en Capas (Layered)	Se organiza internamente en capas horizontales (presentación, lógica de negocio, acceso a datos), donde cada capa solo	Separación por responsabilidades técnicas (no por dominio de negocio); comunicación estrictamente entre capas adyacentes.	Fácil de entender, ampliamente soportado por frameworks (como Spring Boot); buena testabilidad end-to-end.	El acoplamiento entre módulos de negocio dentro de una misma capa puede seguir siendo alto.	Es el tipo elegido para este taller (Spring Boot + Thymeleaf + PostgreSQL); ideal para proyectos pequeños/medianos con equipos reducidos.

	se comunica con la inmediatamente superior o inferior.				
Monolito Modular	Una sola base de código y un solo despliegue, pero organizada internamente en módulos independientes por dominio de negocio (por ejemplo, órdenes, inventario, envíos), con fronteras y contratos bien definidos entre ellos.	Bajo acoplamiento entre módulos, alta cohesión interna; cada módulo puede tener su propia lógica y hasta su propio esquema de datos dentro de la misma base de datos.	Facilita que equipos medianos trabajen en paralelo sin pisarse tanto; prepara el camino para una futura migración a microservicios si se necesita.	Requiere más disciplina de diseño desde el inicio; si los límites entre módulos no se respetan, se pierde la ventaja frente al monolito en capas.	Empresas como Shopify, cuando el negocio tiene varios dominios claros pero aún no se justifica separar en servicios independientes.
Monolito Distribuido	Se despliega como varios procesos o instancias que se comunican por red (HTTP, colas, etc.), pero siguen compartiendo una sola base de datos y un ciclo de despliegue conjunto, o dependen tan fuertemente entre sí que deben desplegarse todos juntos.	Combina la complejidad operativa de un sistema distribuido (red, latencia, fallos parciales) con el acoplamiento de un monolito (no se puede desplegar ni versionar cada parte por separado).	Ninguna ventaja real; suele ser el resultado de una migración a microservicios mal ejecutada.	Se obtienen las desventajas de ambos mundos: la complejidad de lo distribuido sin la independencia de los microservicios.	Es un antipatrón; se debe evitar. Suele aparecer cuando se separan servicios sin antes desacoplar bien los módulos y la base de datos.
Monolito Frontend/Backend separados (2 niveles)	El frontend y el backend son dos aplicaciones monolíticas independientes	Dos bases de código y dos despliegues, pero cada uno sigue siendo	Permite que equipos de frontend y backend trabajen y evolucionen	Introduce comunicación por red (mayor latencia, necesidad de	Aplicaciones donde el frontend (por ejemplo, una SPA) necesita evolucionar a un ritmo distinto al

	(por ejemplo, un backend Spring Boot y un frontend aparte), que se comunican por una API, pero cada una se sigue desplegando como una única unidad.	un monolito internamente.	con cierta independencia; facilita reemplazar el frontend sin tocar el backend.	manejar versiones de API) que un monolito de un solo proceso no tendría.	backend, sin llegar a microservicios.
Big Ball of Mud	No es un tipo de monolito planeado, sino el resultado de un monolito (de cualquier tipo) sin disciplina de diseño: código sin capas claras, con dependencias cruzadas descontroladas.	Alto acoplamiento, baja cohesión, cualquier módulo puede llamar a cualquier otro sin restricciones.	Ninguna; es considerado un antipatrón.	Muy difícil de mantener, probar o escalar en equipo; los cambios pequeños tienen efectos impredecibles.	Nunca se elige a propósito; se debe prevenir manteniendo disciplina en capas/módulos desde el inicio (como se hace en este proyecto).

Casos de uso

I. ¿Cuándo usarlo?

Para usar todo el potencial de esto lo ideal sería usarlo en proyectos pequeños o medianos mas no proyectos grandes, de la misma forma manejar equipos pequeños, aplicaciones donde importe más construir el sistema rápido y que sea fácil de operar, aunque eso signifique que no se pueda escalar cada módulo de forma independiente

II. ¿Cuándo no?

No debería usarse en sistemas que cuenten una gran necesidad de escalar sus componentes de forma independiente, que requieran equipos muy grandes trabajando en paralelo, o cuando el negocio tiene partes muy distintas entre sí y cada una necesita poder actualizarse por separado, sin depender de las demás.

Casos de aplicación (ejemplos reales en la industria)

A día de hoy, hay empresas grandes que siguen usando arquitectura monolítica en sus sistemas a pesar de que sea algo más “tradicional” e incluso pueda ser considerado anticuado, entre esas podemos encontrar por ejemplo a Shopify, la cual tiene su plataforma sobre un monolito modular, o sea una sola base de código organizada internamente por dominios de negocio como órdenes, envíos e inventario. Gracias a esto la empresa logra procesar más de 30 terabytes de datos por minuto y recibir más de 32 millones de solicitudes por minuto en fechas de alta demanda como el Black Friday.

Otro gran ejemplo viene siendo Stack Overflow, esta atiende a millones de desarrolladores todos los días utilizando solamente 9 servidores web. Toda la plataforma corre como un monolito, apoyándose en Redis para el manejo de caché y en Elasticsearch para las búsquedas. Los dos ejemplos anteriores demuestran que una arquitectura monolítica bien diseñada, puede soportar cargas de trabajo bastante altas sin necesidad de recurrir a otro tipo de arquitectura.

Investigación de Stack Asignado

A. Front: JSF – Thymeleaf

Definición clara (qué es y qué no es).

- **JSF (JavaServer Faces):** es una herramienta de Java que sirve para construir las pantallas de una aplicación web usando piezas ya armadas, como botones, tablas y formularios, en lugar de escribir todo el HTML a mano.
- **Thymeleaf:** Herramienta más sencilla usada para armar las páginas HTML de una aplicación web, insertando datos que vienen desde el backend. No es un framework completo como JSF, sino que se apoya en Spring Boot para hacer el trabajo pesado, solo se encuentra encargada de la parte visual.

Características principales.

- **JSF:**
 - Trabaja con componentes que "recuerdan" su estado mientras el usuario interactúa con la página.
 - Sigue un orden de pasos fijo cada vez que se hace una petición.
 - Trae componentes ya armados listos para usar.
 - Separa claramente la parte visual de la lógica del negocio.
- **Thymeleaf :**
 - Sus archivos siguen siendo HTML normal, así que se pueden abrir directamente en el navegador y verse bien, aunque no haya un servidor corriendo detrás.
 - Se conecta fácilmente con los datos que vienen desde Spring Boot, sin necesitar mucho código adicional.
 - Permite mostrar listas, condiciones y textos dinámicos directamente dentro del HTML.

- Es fácil de leer, porque no se aleja mucho de un HTML común.

Historia y evolución.

JSF salió en el 2004 como parte de la plataforma Java EE, buscando que las empresas tuvieran una forma estándar de construir interfaces web en Java. Con el paso del tiempo, fueron apareciendo herramientas más modernas y su uso fue bajando poco a poco, a pesar de ello, se sigue usando todavía en empresas grandes que tienen sistemas antiguos. Por otro lado, Thymeleaf apareció después, en el 2011, este se volvió la opción más usada para construir vistas dentro de proyectos hechos con Spring Boot, prácticamente reemplazando a JSF en los proyectos nuevos.

Ventajas y desventajas.

- **JSF:** De ventaja encontramos que separa bien la parte visual de la lógica del negocio y trae muchos componentes ya hechos, lo cual ahorra tiempo. Como desventaja se tiene que es más difícil de aprender y que cada vez se usa menos en proyectos nuevos.
- **Thymeleaf:** Su ventaja es que es fácil de leer porque sigue siendo HTML normal, lo que facilita trabajar entre quien hace el diseño y quien hace la lógica. Su desventaja es que su forma de escribir código dentro del HTML puede ser confusa al principio y si se abusa de ella las páginas se vuelven difíciles de entender.

Casos de uso (cuándo usarlo y cuándo no).

- **JSF:** Es útil de usar cuando ya se está trabajando con Java EE o se debe mantener un sistema antiguo hecho con esta tecnología. No conviene usarlo en un proyecto nuevo si se busca algo más moderno o sencillo de mantener.
- **Thymeleaf:** Conviene usarlo cuando el proyecto ya está hecho en Spring Boot y solo se necesita mostrar información en pantallas simples. No conviene usarlo si la aplicación necesita una interfaz muy interactiva.

Casos de aplicación (ejemplos reales en la industria).

- **JSF:** Se ha usado en producción en empresas grandes y bancos como Credit Suisse, Deutsche Bank, FedEx o Apple, además de estar integrado en servidores empresariales muy usados como IBM WebSphere y Oracle WebLogic.
- **Thymeleaf:** Al ser el motor recomendado por el propio equipo de Spring, se usa en la gran mayoría de aplicaciones nuevas que se construyen con Spring Boot en la industria hoy en día como JHipster.

Relación entre el estilo y las tecnologías seleccionadas.

Tanto JSF como Thymeleaf encajan bien con el monolito porque las pantallas viven dentro de la misma aplicación que la lógica de negocio. No necesitan comunicarse por internet ni por servicios externos, simplemente llaman directamente a las funciones del backend, que es justo el tipo de comunicación que les asignaron para su proyecto.

Que tan común es el stack designado (relación entre tecnologías).

No es muy común el usar JSF y Thymeleaf al tiempo en un proyecto real, porque ambas cumplen la misma función de mostrar las pantallas. Lo normal es escoger una de las dos, dependiendo de si el proyecto es antiguo (JSF) o nuevo (Thymeleaf con Spring Boot).

B. Back: Spring Boot/ Java

Definición clara (qué es y qué no es).

- **Spring Boot:** Framework construido sobre Java (específicamente sobre el Spring Framework) el cual facilita crear aplicaciones listas para producción, sin tener que configurar todo de forma manual. No es un lenguaje ni reemplaza a Java, más bien es una herramienta que ahorra trabajo repetitivo (como la configuración, conexión a base de datos, servidor, entre otros) para que uno se enfoque en la lógica del negocio.
- **Java:** Lenguaje de programación sobre el que corre todo el backend, lenguaje con el que se escribe la lógica del sistema.

Características principales.

- Configura automáticamente muchas cosas por defecto en lugar de que se tenga que hacer a mano.
- Trae un servidor incluido (como Tomcat), así que no hay que instalar uno aparte para correr la aplicación.
- Permite crear un proyecto nuevo fácil y rápido usando una herramienta llamada Spring Initializr.
- Reduce el código repetitivo que se escribe en Java puro.

Historia y evolución.

Spring Framework nació en 2002, propósito desde un comienzo fue hacer más simple el desarrollo de aplicaciones empresariales en Java. Al pasar el tiempo, configurar Spring se volvió complicado, así que en 2014 salió Spring Boot como una forma de usar Spring sin configuración manual. Desde ese momento se volvió bastante popular, primero para construir APIs y servicios web, luego también para microservicios, llegando a ser uno de los frameworks más usados en el desarrollo empresarial con Java.

Ventajas y desventajas.

- **Ventajas:**
 - Ahorra mucho tiempo de configuración.
 - Ayuda a escribir menos código innecesario.
 - Facilita crear una aplicación que funcione sin depender de instalar muchas cosas aparte.
- **Desventajas:**
 - Al configurar tanto de forma automática puede traer en ocasiones algunas dependencias que no se necesitan, lo que hace más pesada la aplicación.

- En caso de necesitar o querer personalizar algo que se sale de lo estándar, puede volverse más complicado de lo normal.

Casos de uso (cuándo usarlo y cuándo no).

Es mejor usarlo cuando se quiere desarrollar rápido una aplicación backend, mas no es útil usarlo si se requiere un control muy detallado sobre cada parte de la configuración.

Casos de aplicación (ejemplos reales en la industria).

Es usado por varias de las empresas tecnológicas más grandes del mundo, entre las cuales se encuentran:

- Netflix la cual tiene una gran parte de su sistema de microservicios con Spring Boot, lo que le permite atender a sus miles de usuarios y procesar millones de peticiones.
- Amazon que usa Spring Boot en varios de sus servicios de backend, incluyendo el seguimiento de muchos pedidos en tiempo real al mismo tiempo.
- Airbnb lo usa para su sistema de reservas, aprovechando que Spring Boot permite desplegar actualizaciones rápido y escalar según la demanda mundial.

Empresas como Google, PayPal y Uber también han construido partes de sus plataformas sobre Spring Boot.

Relación entre el estilo y las tecnologías seleccionadas.

Spring Boot encaja bien con la arquitectura monolítica, ya que este permite tener toda la aplicación, desde las vistas hasta la conexión con la base de datos, corriendo dentro de un mismo proceso y despliegue, comunicando todo con llamados directos a funciones, sin servicios externos.

Que tan común es el stack diseñado (relación entre tecnologías).

Es muy común, Spring Boot con Java es una de las combinaciones más usadas hoy en día para construir backends en la industria.

C. Base de Datos: PostgreSQL/SQL

Definición clara (qué es y qué no es).

- **PostgreSQL:** Sistema de base de datos que usa SQL para guardar y organizar información en tablas relacionadas entre sí.
- **SQL:** Lenguaje utilizado para realizar consultas y manejar datos en una base de datos relacional, no es una base de datos en sí misma, sino el lenguaje que se usa en una.

Características principales.

- Guarda la información en tablas relacionadas entre sí, siguiendo el modelo relacional.
- Garantiza que las transacciones se completen por completo o no se apliquen (ACID), evitando datos a medias si algo falla.

- Es de código abierto y gratuito.
- Permite crear tipos de datos, funciones y reglas propias según lo que necesite el proyecto.
- Puede combinar datos tradicionales en tablas con formatos más flexibles como JSON.

Historia y evolución.

PostgreSQL se creó en 1986 en la Universidad de Berkeley como el proyecto POSTGRES, que Michael Stonebraker creó. La primera versión pública de este salió en 1997, desde entonces ha tenido casi 40 años de desarrollo continuo por parte de una comunidad abierta, lo que la ha vuelto una de las bases de datos relacionales más usadas del mundo.

Ventajas y desventajas.

- **Ventajas:**
 - Es gratuito y de código abierto.
 - Es confiable y mantiene la integridad de los datos.
 - Permite personalizar bastante su funcionamiento (tipos de datos, funciones propias, etc.).
 - Soporta bien cargas de trabajo grandes.
- **Desventajas:**
 - Administrarlo puede ser más complicado que otras bases de datos más simples.
 - En consultas de lectura muy sencillas puede ser un poco más lento que alternativas como MySQL.

Casos de uso (cuándo usarlo y cuándo no).

Debería usarse en momentos en los que se necesite dar manejo de forma confiable a datos relacionados entre sí, como en sistemas financieros o de salud. No es conveniente usarlo si solo se necesita guardar datos simples sin relaciones, en estos casos una base de datos más sencilla o ligera sería suficiente.

Casos de aplicación (ejemplos reales en la industria).

Empresas como Instagram, Spotify, Apple y Reddit usan PostgreSQL para manejar los grandes volúmenes de datos de sus usuarios.

Relación entre el estilo y las tecnologías seleccionadas.

Para el taller PostgreSQL guardará todos los datos del ejemplo práctico planteado en un solo lugar, al cual accederá directamente el backend de Spring Boot dentro del mismo despliegue.

Que tan común es el stack designado (relación entre tecnologías).

El uso de ambos (Spring Boot con PostgreSQL) es una combinación común para construir backends.

D. Protocolo de integración: Llamado a Funciones

Definición clara (qué es y qué no es).

“Llamado a funciones” es cuando una parte del programa necesita algo de otra parte de este mismo y directamente “llama” o invoca a esa función o método dentro del programa. Esto no es lo mismo que una llamada remota (como sería en REST o en gRPC) en donde dos programas separados se comunican a través de la red, en este caso todo ocurre dentro de un mismo proceso, en la misma memoria.

Características principales.

- Ocurre dentro del mismo proceso, sin pasar por la red.
- Es sincrónico, quien llama a la función debe esperar a que esta termine para continuar.
- No necesita convertir los datos a otro formato para enviarlos, esto ya que todo se encuentra en la misma memoria.
- El lenguaje de programación revisa en tiempo de compilación que los tipos de datos y los nombres de las funciones sean los correctos.

Historia y evolución.

El llamado a funciones se podría decir que nació con la misma programación, cuando los lenguajes de programación empezaron a organizar el código en funciones y procedimientos reutilizables en lugar de escribir todo de corrido. Con el tiempo, cuando los sistemas empezaron a distribuirse, surgieron formas como el RPC, REST o gRPC. A pesar de eso, el llamado a funciones nunca dejó de ser la forma básica y más directa de comunicación entre las partes del código.

Ventajas y desventajas.

- **Ventajas:**
 - Mucho más rápido al no existir retraso de red.
 - Más fácil de programar y de depurar usando las herramientas normales del lenguaje.
 - No requiere configuración adicional, como servidores o protocolos de red.
- **Desventajas:**
 - Si un componente falla puede afectar directamente a los demás, debido a que se encuentran en el mismo proceso.
 - No permite que las distintas partes del sistema se ejecuten o escalen por separado en máquinas diferentes.

Casos de uso (cuándo usarlo y cuándo no).

Es útil usarlo cuando todo el sistema corre en un mismo despliegue, no cuando se necesita que distintas partes del sistema estén en servidores diferentes.

Casos de aplicación (ejemplos reales en la industria).

Cualquier aplicación monolítica en producción usa llamados a funciones internamente entre sus capas.

Relación entre el estilo y las tecnologías seleccionadas.

Es lo más normal que existe en la forma de comunicación de los componentes de un monolito, ya que todos viven en el mismo proceso.

Que tan común es el stack designado (relación entre tecnologías).

Es la forma de comunicación más básica y común dentro de cualquier aplicación monolítica, o incluso en otro tipo de arquitecturas, sin importar el lenguaje o framework que se use.

ANÁLISIS ARQUITECTÓNICO

Matriz de Atributos de Calidad vs. Estilo (Monolito en capas)

Tabla 2

Atributo de Calidad (ISO 25010)	¿Cómo lo soporta o limita el estilo?	Nivel de soporte	Justificación
Adecuación Funcional (functional suitability)	Es neutral frente al estilo: depende de que la lógica de negocio esté bien implementada, no de cómo se despliegue la aplicación.	Neutral	El monolito no impide ni favorece que el sistema cumpla los requisitos funcionales; eso depende del diseño de cada módulo.
Eficiencia de Desempeño (performance efficiency)	Lo soporta bien, ya que la comunicación entre capas es por llamado directo a funciones dentro del mismo proceso, sin latencia de red ni serialización.	Alto	Al no existir llamadas remotas entre Controller, Service y Repository, las operaciones se resuelven más rápido que en un sistema distribuido.
Compatibilidad (compatibility)	Lo limita, porque no está pensado para coexistir o intercambiar datos fácilmente con otros sistemas externos sin capas adicionales.	Bajo/Medio	Integrarse con otros sistemas (como otros bancos de sangre de la red RIBAS) requeriría exponer una API o servicio adicional, algo que el estilo base no contempla.
Capacidad de Interacción (interaction capability / usabilidad)	Es neutral, depende más de la capa de presentación (Thymeleaf/JSF) que del estilo arquitectónico en sí.	Neutral	El monolito no interviene directamente en qué tan fácil o agradable es el sistema para el usuario final.
Confiabilidad (reliability)	Lo limita, ya que al compartir el mismo proceso y la misma base de datos, un fallo en un módulo puede afectar la disponibilidad de todo el sistema.	Bajo/Medio	No hay aislamiento de fallos entre componentes; un error no controlado en una capa puede propagarse y tumbar toda la aplicación.
Seguridad (security)	Lo soporta de forma aceptable, al tener un único punto de despliegue	Medio/Alto	Menor superficie de ataque que en microservicios (menos puntos de red

	es más fácil centralizar la autenticación, autorización y auditoría.		expuestos), aunque una vulnerabilidad puede comprometer toda la aplicación a la vez.
Mantenibilidad (maintainability)	Depende del orden interno: si se respeta la separación en capas, es manejable; si no, tiende al Big Ball of Mud.	Medio	El estilo no garantiza mantenibilidad por sí solo, la favorece solo si el equipo respeta la organización en capas definida para el proyecto.
Portabilidad (portability)	Lo soporta razonablemente bien, ya que al ser un solo paquete desplegable (por ejemplo, un .jar de Spring Boot), es fácil moverlo entre distintos entornos.	Medio/Alto	Un solo artefacto de despliegue simplifica llevar la aplicación de un ambiente a otro, aunque sigue dependiendo de una única base de datos configurada externamente.
Seguridad Operacional (safety)	Es neutral frente al estilo: depende del dominio del sistema (en este caso, datos sensibles de bancos de sangre) y de los controles implementados, no de si es monolito o no.	Neutral	El estilo arquitectónico no determina por sí mismo el riesgo de daño a personas o datos críticos; eso se gestiona con reglas de negocio, validaciones y controles de acceso propios de la aplicación.

Matriz de Análisis de Principios vs. Estilo (Monolito en capas)

Tabla 3

Principios vs Estilo

Principio	¿Cómo se cumple o incumple en el estilo?	Nivel de cumplimiento
S – Responsabilidad Única (SRP)	Se puede cumplir si cada capa (presentación, negocio, datos) y cada clase tiene una única responsabilidad; el estilo no lo obliga, pero lo facilita al permitir capas bien delimitadas.	Depende del diseño
O – Abierto/Cerrado (OCP)	Es más difícil de aplicar de forma estricta, ya que un monolito tiende a modificar directamente las clases existentes en lugar de extenderlas, por la presión de entregar rápido.	Medio
L – Sustitución de Liskov (LSP)	No depende del estilo arquitectónico sino del diseño orientado a objetos interno; es igual de aplicable en un monolito que en cualquier otro estilo.	Neutral

I – Segregación de Interfaces (ISP)	Se puede respetar fácilmente al definir interfaces específicas por capa (por ejemplo, interfaces de repositorio por entidad), el monolito no lo impide.	Alto
D – Inversión de Dependencias (DIP)	Spring Boot lo favorece de forma nativa mediante inyección de dependencias, incluso dentro de un monolito las capas pueden depender de abstracciones.	Alto
KISS (Keep It Simple)	El monolito en capas es, en sí mismo, una decisión "simple": un solo despliegue, sin redes ni protocolos adicionales que administrar.	Alto
DRY (Don't Repeat Yourself)	Es más fácil de lograr que en microservicios, porque al compartir una sola base de código, la lógica común se puede reutilizar directamente sin duplicar servicios.	Alto
YAGNI (You Aren't Gonna Need It)	Se alinea bien con el estilo, ya que un monolito evita construir infraestructura de comunicación distribuida que el proyecto (pequeño/mediano) no necesita todavía.	Alto
PoLA (Principio de Mínima Sorpresa)	Se cumple si las capas siguen convenciones claras (por ejemplo, Controller → Service → Repository de Spring Boot), lo cual es un patrón muy conocido y esperado por cualquier desarrollador Java.	Alto
Ley de Demeter	Es más fácil de romper en un monolito, porque al tener todo en el mismo proceso, es tentador que una capa acceda directamente a objetos internos de otra en lugar de pasar por su interfaz.	Medio (riesgo si no se disciplina)
STUPID (anti-patrones a evitar)	El riesgo de caer en Acoplamiento fuerte (Tight Coupling) y en Duplicación es mayor si no se respeta la separación en capas; el monolito no obliga a evitarlo, depende de la disciplina del equipo.	Medio (depende del diseño)
Composición sobre Herencia	No depende del estilo arquitectónico, sino del diseño de las clases dentro de cada capa; es igual de aplicable en el monolito que en cualquier otro estilo.	Neutral

Matriz de Análisis de Tácticas vs. Estilo y Stack

Tabla 4

Táctica arquitectónica	Atributo	Aplicación en el estilo (Monolito en capas)	Aplicación en el stack asignado	Justificación para un ADR
Validación de reglas de negocio centralizada en la capa de servicio.	Adecuación Funcional	Toda la lógica de negocio vive en una sola capa (Service), sin fragmentarse	Clases @Service de Spring Boot concentran las reglas del dominio (por ejemplo, validar compatibilidad de tipo de sangre).	Justifica concentrar las reglas de negocio en la capa Service y no en los Controllers ni

		entre servicios distintos.		en las vistas Thymeleaf, para que la lógica no quede dispersa.
Comunicación síncrona por llamado directo a funciones + caché de datos.	Eficiencia de Desempeño	Las capas se comunican dentro del mismo proceso, sin latencia de red.	Llamado a funciones entre Controller → Service → Repository; caché opcional con Spring Cache sobre PostgreSQL.	Justifica no introducir un protocolo de red (REST/gRPC) entre capas internas, ya que agregaría latencia y complejidad innecesarias para el alcance del proyecto.
Exposición de una capa de integración (API) desacoplada del núcleo del monolito.	Compatibilidad	El monolito no expone servicios externos de forma nativa; requiere una capa adicional para interoperar.	Un Controller REST adicional en Spring Boot podría exponer endpoints para que otros nodos de la red RIBAS consulten datos.	Justifica, si se requiere compatibilidad con otros sistemas, añadir una capa de API explícita en vez de modificar el estilo base del monolito.
Separación estricta entre la capa de presentación y la lógica de negocio.	Capacidad de Interacción	La capa de presentación solo muestra datos ya procesados, sin contener lógica de negocio.	Thymeleaf/JSF consume modelos ya preparados por los Services de Spring Boot.	Justifica el uso de Thymeleaf sobre Spring Boot, evitando mezclar lógica de negocio dentro de las plantillas HTML, lo que mejora la interacción y el mantenimiento de las vistas.

Manejo centralizado de errores y transacciones ACID.	Confiabilidad	Al compartir un solo proceso y una sola base de datos, es más fácil garantizar consistencia, pero un fallo puede afectar todo el sistema.	Transacciones gestionadas con <code>@Transactional</code> de Spring sobre PostgreSQL; manejo global de excepciones (<code>@ControllerAdvice</code>)	Justifica elegir PostgreSQL, con soporte ACID, y un manejo de errores centralizado, priorizando la consistencia de datos críticos (bancos de sangre) sobre la disponibilidad parcial.
Autenticación y autorización centralizadas en un único punto de entrada.	Seguridad	Al tener un solo despliegue, es más fácil aplicar una política de seguridad uniforme a toda la aplicación.	Spring Security integrado en el mismo proyecto Spring Boot, controlando el acceso antes de llegar a los Controllers.	Justifica usar Spring Security de forma centralizada en vez de manejar seguridad por módulo, reduciendo el riesgo de dejar rutas desprotegidas.
Encapsulamiento por capas y uso de interfaces/abstracciones	Mantenibilidad	Cada capa expone solo lo necesario a la capa superior, reduciendo el acoplamiento	Interfaces de repositorio de Spring Data JPA sobre PostgreSQL; Services que dependen de abstracciones, no de implementaciones concretas.	Justifica usar Spring Data JPA en lugar de acceso directo con JDBC, priorizando mantenibilidad y facilidad de cambio sobre control fino del SQL.
Autoconfiguración y empaquetado como artefacto único desplegable.	Portabilidad	El monolito se construye y despliega como una sola unidad, fácil de	Spring Boot genera un único .jar autocontenido con servidor embebido (Tomcat).	Justifica el uso de Spring Boot frente a un despliegue manual en servidor

		mover entre entornos.		externo, ya que el artefacto único facilita portar la aplicación entre ambientes de desarrollo, pruebas y producción.
Reglas de negocio y validaciones estrictas antes de persistir datos críticos.	Seguridad Operacional (Safety)	Al concentrarse toda la lógica en un solo lugar, es más fácil imponer validaciones consistentes antes de guardar información sensible.	Validaciones en la capa Service antes de invocar al Repository (por ejemplo, verificar compatibilidad sanguínea antes de registrar una transfusión).	Justifica centralizar las validaciones críticas en la capa de negocio del monolito, evitando que datos inválidos o peligrosos (dado el dominio de bancos de sangre) lleguen a la base de datos.

Matriz de análisis de mercado Laboral vs estilo y stack

Tabla 5

Mercado vs Estilo y Stack

Componente	Estado actual en el mercado	Demanda laboral	Salario de referencia	Proyección
Java	4.º en TIOBE con 7,54 % en septiembre de 2026, su valor mensual más bajo desde 2001; 2.º en PYPL. Cerca del 29 % de los	Entre los tres lenguajes más solicitados en ofertas (junto a Python y JavaScript). Fuerte en banca, sistemas críticos y modernización de plataformas.	Colombia: 4.263.222 COP/mes ($\approx$ 51,2 M COP/año, cálculo propio) según Indeed. EE. UU.: $\approx$ US\$120.300/año (Glassdoor);	Demanda estable - cuota relativa a la baja por la presión de Python. Puede llegar a depender de migración de versiones con la terminación del soporte premier de Java 17 en

	respondientes de Stack Overflow 2025 lo usó. Java 25 es la versión LTS vigente.		mediana de US\$160.000/año entre profesionales que usan Java (Stack Overflow 2025).	septiembre de 2026.
Spring Boot	Framework web Java más adoptado: 14,7 % de todos los desarrolladores (Stack Overflow 2025) con 53,7 % de admiración; Spring lidera con 65 % entre desarrolladores Java (JetBrains 2025). Spring Boot 4.0 salió en noviembre de 2025.	Requisito habitual en ofertas backend Java. Complementos frecuentes: Hibernate, SQL, PostgreSQL, Docker, Maven.	EE. UU.: US\$117.732/año en promedio (rango típico US\$91.469–153.142; percentil 90: US\$192.720) según Glassdoor; US\$123.333/año según ZipRecruiter. Colombia: sin dato específico; aplica el rango de Java (sección II).	Demanda estable - es el estándar para nuevos backends empresariales. Alternativas como Quarkus o Micronaut ocupan nichos cloud-native.
Thymeleaf	Motor de plantillas habitual en proyectos Spring Boot con renderizado en servidor. No aparece con cifras propias en TIOBE, Stack Overflow ni JetBrains.	Se pide como parte del perfil "Java fullstack", rara vez como habilidad aislada.	Reino Unido: mediana de £77.500/año (unico dato razonable encontrado)	Estable en aplicaciones de servidor - menos visible en ofertas orientadas a SPA (estimación propia).

JSF	En retroceso; se mantiene en sistemas Java EE heredados y suele asociarse a PrimeFaces.	En el Reino Unido solo 5 ofertas permanentes citaron JSF en los 6 meses previos a enero de 2026 (8 el año anterior y 16 dos años antes).	se remunera como parte del rol Java/Spring	Descendente - mercado de mantenimiento de legados. Para un proyecto nuevo, Thymeleaf es la apuesta más empleable. desarrollo sobre configuración detallada.
PostgreSQL	Base de datos más usada en Stack Overflow 2025 (55,6 %), la más admirada (65 %) y la más deseada (46 %), con 15 puntos de ventaja sobre MySQL.	Se exige como complemento en casi cualquier rol backend.	PostgreSQL Developer: ZipRecruiter reporta 123.262 USD de promedio, con la mayoría entre 102.500 y 142.000. Glassdoor da 129.532 USD, pero con solo 5 salarios enviados.	Al aumento - con el mayor crecimiento anual de su historia en la encuesta.
Complementos (Maven, Docker, JPA/Hibernate)	Maven es la herramienta de construcción más usada entre desarrolladores Java (67 %, JetBrains 2025). Docker e Hibernate figuran como habilidades complementarias frecuentes.	Suman empleabilidad sin ser el criterio principal.	Incluido en roles previos, son habilidades que suman empleabilidad pero no es la razón de contrato en la mayoría de los casos	Estable.

DISEÑO EJEMPLO PRÁCTICO

Diagrama de alto nivel (HDL)

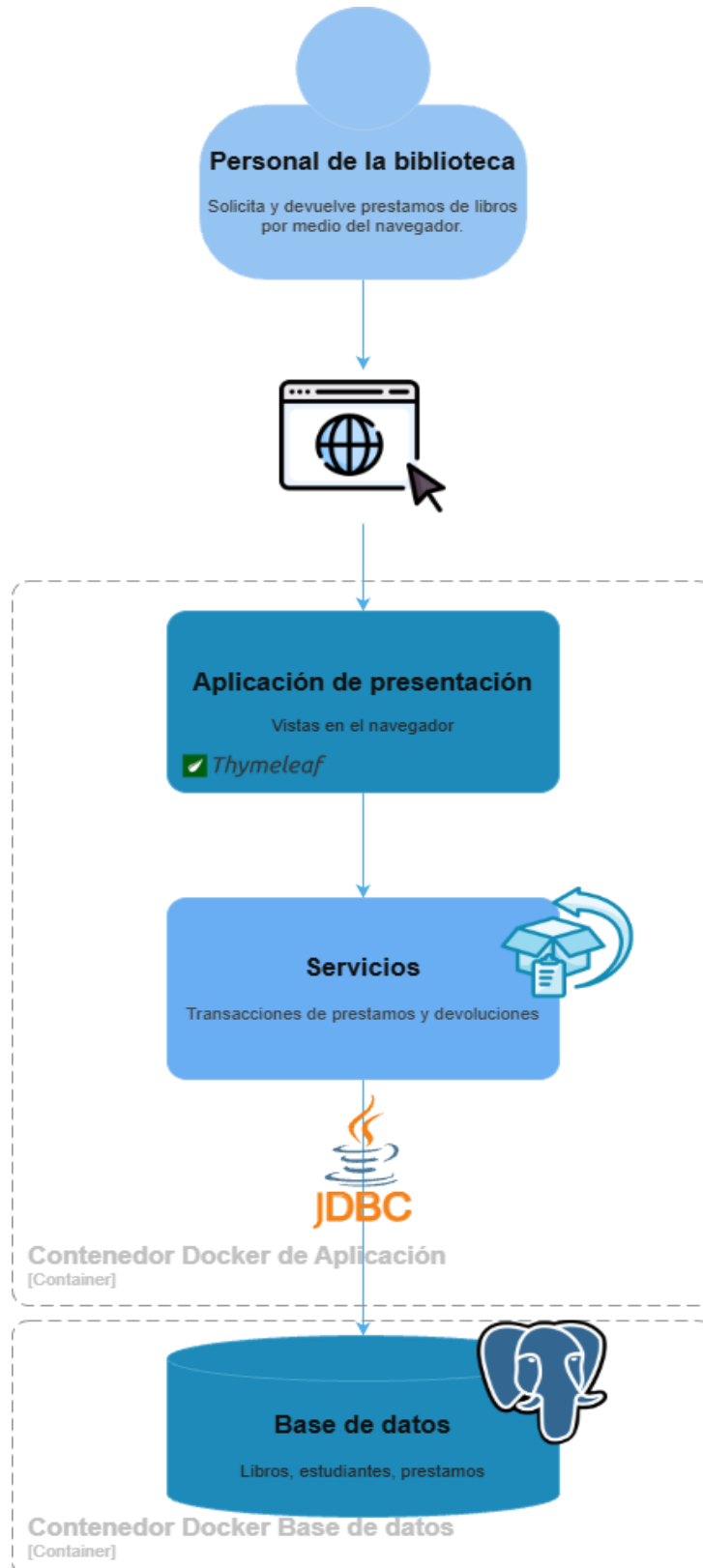


Diagrama de contexto (C1)

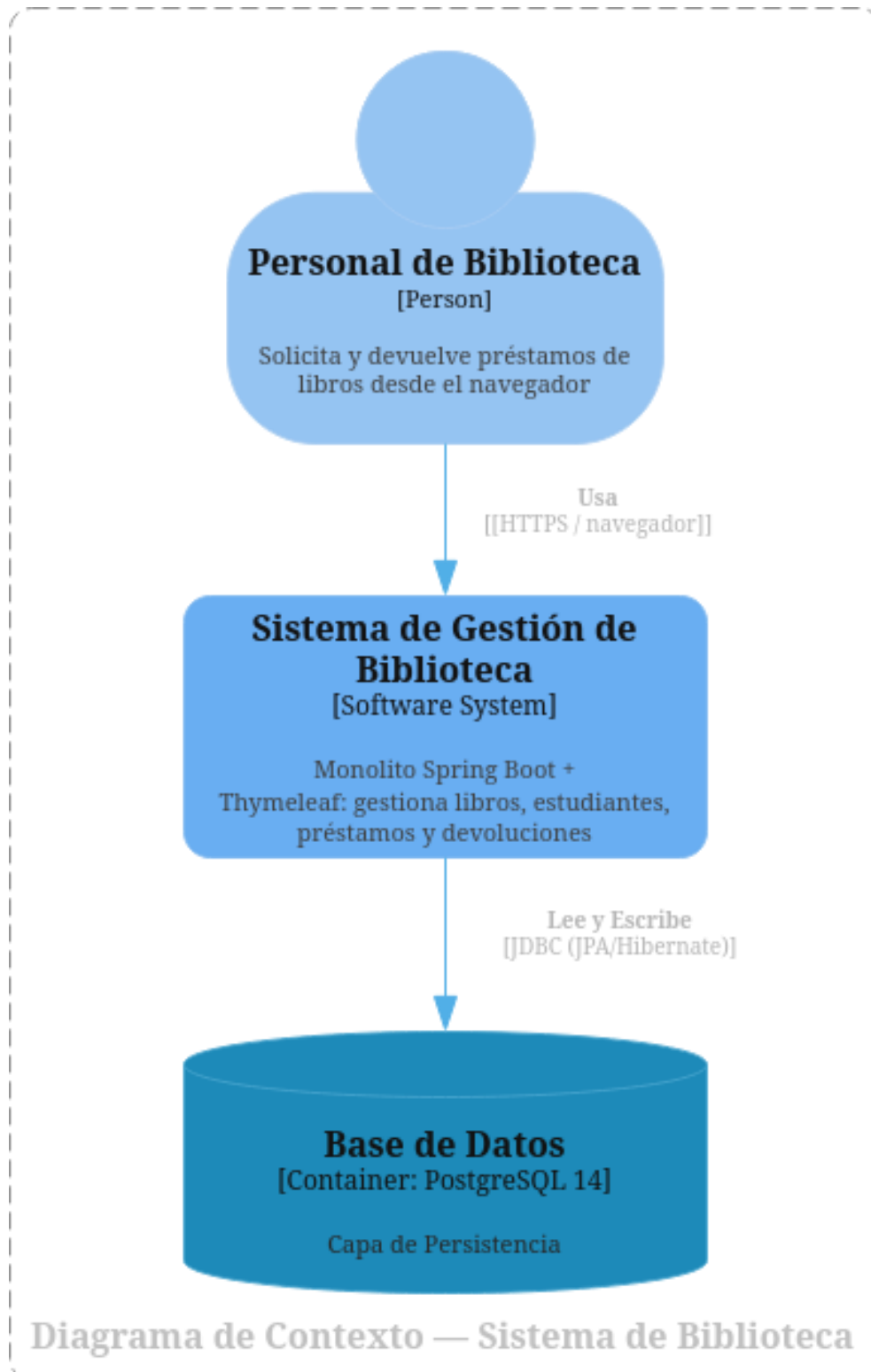


Diagrama de contenedores (C2)

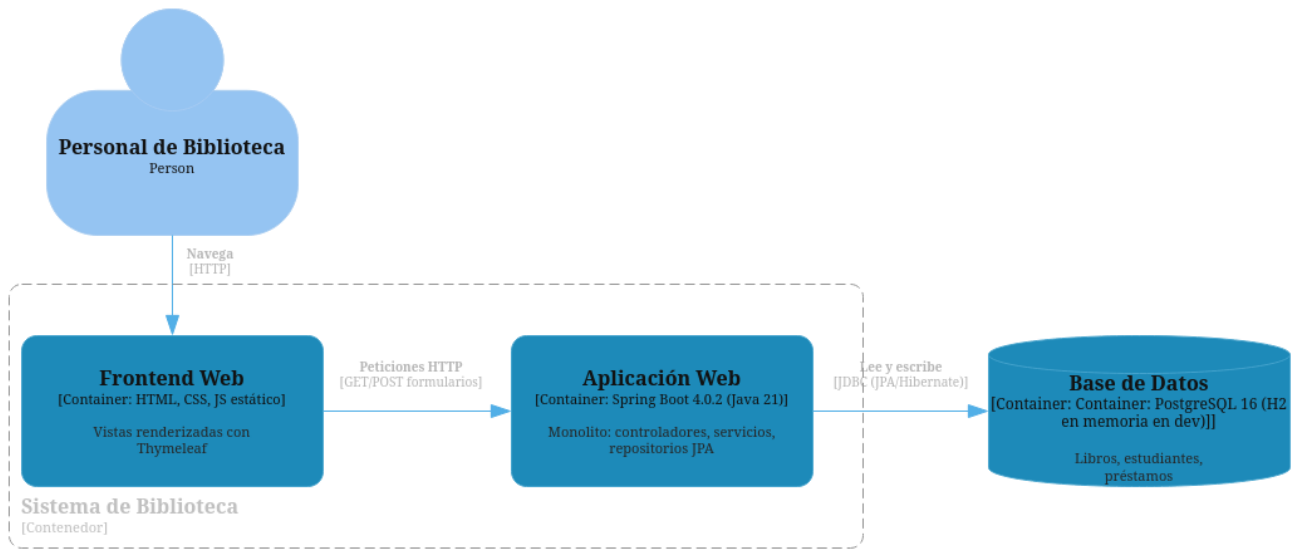


Diagrama dinámico (Préstamo y devolución)

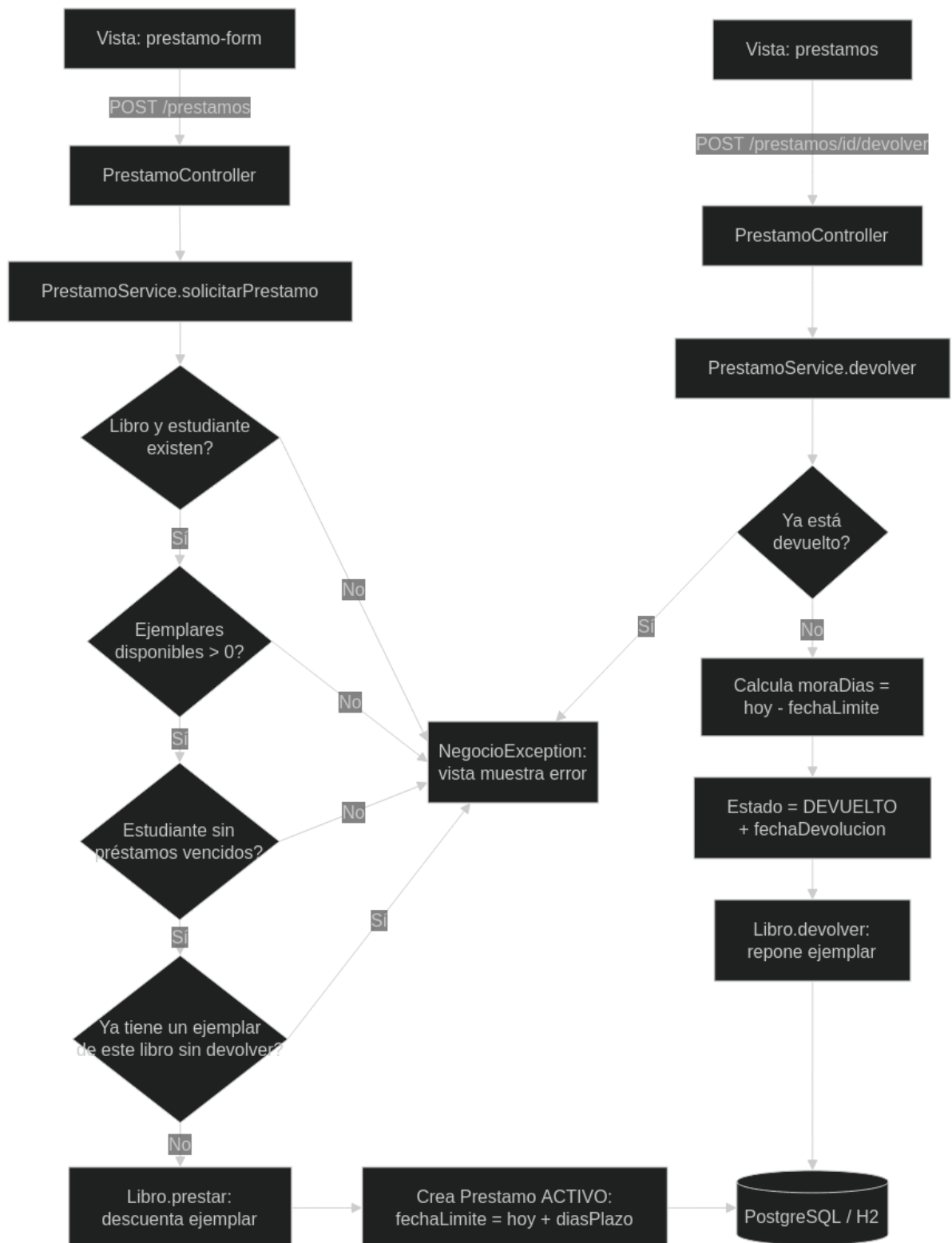
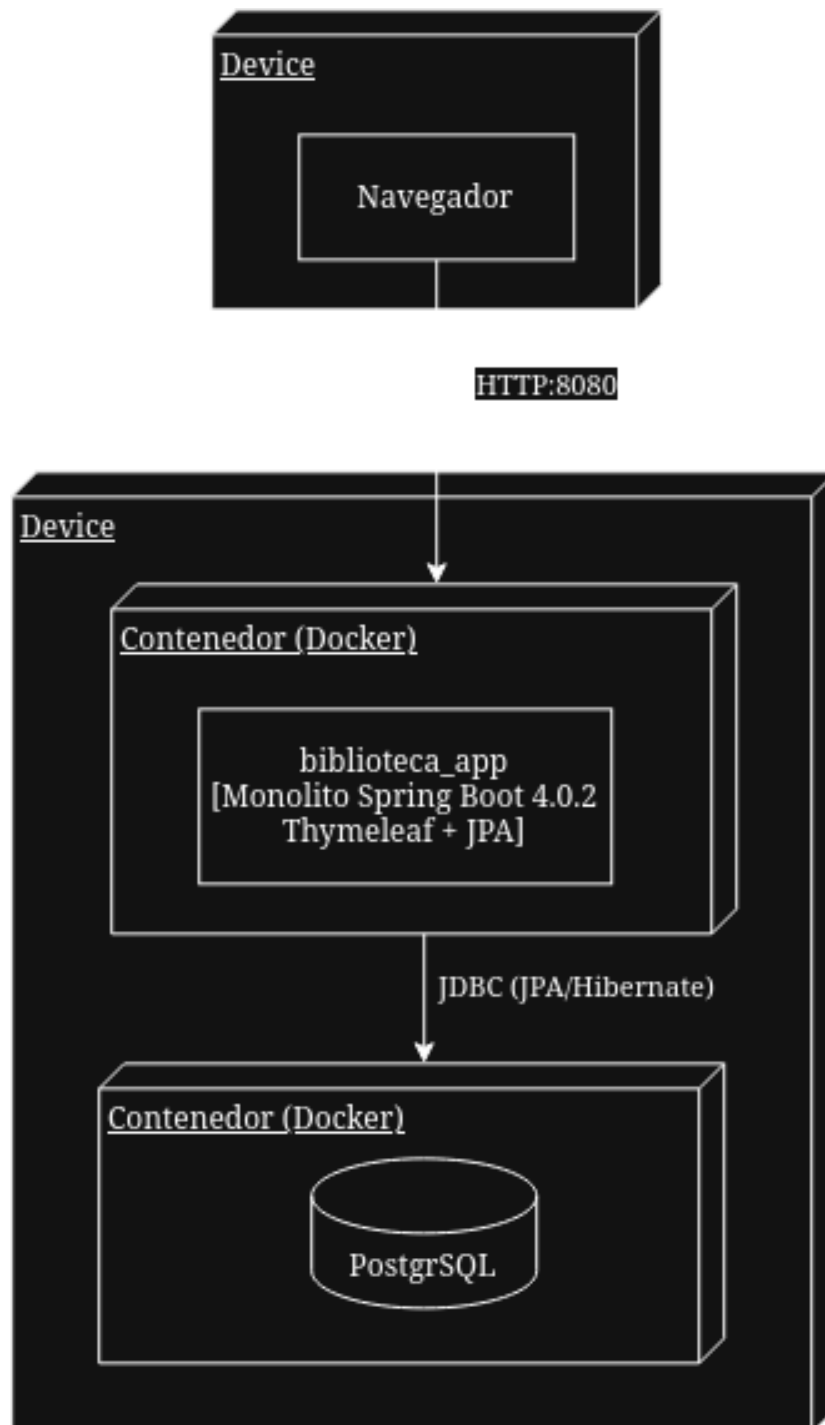


Diagrama de despliegue



LECCIONES APRENDIDAS

A partir de lo que se hizo como grupo en el taller y lo que se asignó, fue posible entender de mejor forma ciertos conceptos que en semestres anteriores no fue posible tener claro en su totalidad, así como simplemente recordar alguno que otro concepto que estaba claro pero simplemente no se recordaba y tener un poco más presente algunos ejemplos de aplicaciones prácticas

Además de ello, dentro de las cosas técnicas, como equipo fue posible identificar que no se venían implementando ciertas cosas de la forma correcta; se reforzó la construcción de diagramas, que ha sido una gran falencia desde el comienzo del semestre, en especial con el modelo C4; se reforzó un poco el cómo asignar adecuadamente los atributos de calidad, a pesar de que todavía puedan existir errores; ayudó a revisar con qué criterio normalmente ya sea en el proyecto o en diferentes materias, se escoge el stack tecnológico y que se está dispuesto a sacrificar con cada decisión

Finalmente, con respecto a el monolito, se identificó la importancia de llevar un orden para no causar un "Big Ball of Mud" y que, a pesar de ser simple en arquitectura, favorecer el rendimiento, el testeado y el costo operativo, limita la escalabilidad, la disponibilidad y la capacidad de modificar, por lo cual hay que ser muy conscientes a la hora de escoger este estilo de acuerdo con el contexto que se tenga.

REFERENCIAS

- Awati, R., & Wigmore, I. (2024, October 16). What is monolithic architecture in software? TechTarget. WhatIs. <https://www.techtarget.com/whatis/definition/monolithic-architecture>
- Coding Shuttle. (2024, September 29). Introduction to Spring Boot. Coding Shuttle. <https://www.codingshuttle.com/spring-boot-handbook/introduction-to-spring-boot/>
- Davis, E. (2025, January 15). Spring Boot success stories: Real-world applications & impact. CodingCops. <https://codingcops.com/spring-boot-success-stories/>
- Dhruv, S. (n.d.). PostgreSQL advantages and disadvantages. Aalpha. Retrieved September 17, 2026, from <https://www.aalpha.net/blog/pros-and-cons-of-using-postgresql-for-application-development/>
- Kohlberg, M. (2023, June 14). Monolithic Architecture – Strengths, weaknesses, and characteristics. Encore. Encore.dev. <https://encore.dev/articles/monolithic-architecture>
- Milanović, M. (2023, July 13). Stack Overflow architecture. Tech World With Milan Newsletter. <https://newsletter.techworld-with-milan.com/p/stack-overflow-architecture>
- Ozkaya, M. (2024, September 3). How Modular Monolithic Architecture Handles Communications Between Modules — In-process method.... Medium. <https://mehmetozkaya.medium.com/how-modular-monolithic-architecture-handles-communications-between-modules-in-process-method-7be34fa920e6>
- Singour, H. (2025, October 9). How Shopify Handles 30TB of Data Every Minute with a monolithic architecture. Medium. <https://medium.com/@himanshusingour7/how-shopify-handles-30tb-of-data-every-minute-with-a-monolithic-architecture-cad54df86955>
- Spring boot framework: Features, benefits, and use cases. (n.d.). StaticMania. Retrieved September 17, 2026, from <https://staticmania.com/blog/spring-boot>
- Srivastava, A. (2025, April 24). All about spring thymeleaf you need to know in 2025. DEV Community. https://dev.to/abhay_srivastava_22/all-about-spring-thymeleaf-you-need-to-know-in-2025-2omd
- Surmann, T. (2025, November 14). Jte vs Thymeleaf — advantages and disadvantages. Medium. <https://bootify.medium.com/jte-vs-thymeleaf-advantages-and-disadvantages-a5d53f4e9fd4>
- What is PostgreSQL? (2021, August 12). OpenLogic. <https://www.openlogic.com/blog/what-is-postgresql>
- Fosspost. (2026). Java statistics 2026 [Releases, JVM usage and market share]. <https://fosspost.org/java-statistics/>
- Glassdoor. (2026, marzo). Java Spring Boot Developer: Average salary & pay trends 2026. https://www.glassdoor.com/Salaries/java-spring-boot-developer-salary-SRCH_KO0,26.htm

Glassdoor. (2026, marzo). Java Spring Developer: Average salary & pay trends 2026.
https://www.glassdoor.com/Salaries/java-spring-developer-salary-SRCH_KO0,21.htm

Indeed. (2026). Sueldo de Desarrollador/a java en Colombia.
<https://co.indeed.com/career/desarrollador-java/salaries>

Indeed. (2026, 31 de agosto). Sueldo de Desarrollador/a junior en Colombia.
<https://co.indeed.com/career/desarrollador-junior/salaries>

IT Jobs Watch. (2026, enero). JSF job trends [Reino Unido].
<https://www.itjobswatch.co.uk/jobs/uk/jsf.do>

JetBrains. (2025). The state of Java 2025. <https://lp.jetbrains.com/the-state-of-java-2025/>

Motion Recruitment. (2026). 2026 Java developer salary guide.
<https://motionrecruitment.com/it-salary/java>

PayScale. (2026). Java Developer with Spring skills salary.
https://www.payscale.com/research/US/Job=Java_Developer/Salary/488613ae/Spring

Rockstar Developer University. (2026, 20 de julio). Software developer employment statistics 2026 [con datos del BLS]. <https://rockstardeveloperuniversity.com/software-developer-employment-statistics/>

Salary.com. (2026, 1 de agosto). Java Spring MVC Developer salary.
<https://www.salary.com/research/salary/listing/java-spring-mvc-developer-salary>

Second Talent. (2026, 14 de septiembre). Java statistics: Adoption, usage, and future trends [2026]. <https://www.secondtalent.com/resources/domain-java-statistics/>

Stack Overflow. (2025). 2025 Stack Overflow developer survey: Technology.
<https://survey.stackoverflow.co/2025/technology>

Talently. (2026). Cuánto gana un programador Java en Colombia.
<https://talently.tech/herramientas/colombia/salario/skill/java>

TechRepublic. (2026, septiembre). TIOBE Index September 2026: Julia nears top 20.
<https://www.techrepublic.com/article/news-tiobe-september-2026-julia-nears-top-20/>

Vaadin. (2026). Most popular Java frameworks in 2026. <https://vaadin.com/blog/most-popular-java-frameworks-2026>

ZipRecruiter. (2026, marzo). Java Spring Developer salary.
<https://www.ziprecruiter.com/Salaries/Java-Spring-Developer-Salary>